

6.To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```
GNU nano 8.7.1 skill6.c
#include <stdlib.h>

int main() {
    char command[100];

    printf("Enter command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        char *args[20];
        int i = 0;

        char *token = strtok(command, " ");

        while (token != NULL && i < 19) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }

        args[i] = NULL;

        execvp(args[0], args);

        perror("execvp failed");
    }
}
```

```

bindu@Bindusree:~$ nano skill6.c
bindu@Bindusree:~$ gcc skill6.c -o skill6
bindu@Bindusree:~$ ./skill6
Enter command: ls
Hello.save          hello.c             practical2.c
HellofromParent.c   hello.c.save        practical5b
MultiChild          input.txt           practical5b.c
MultiChild.c         input.txt           priority.c
Week1_1             input.txt.save      priority.c.save
Week1_1.c           input.txt.save.1    processid
Week1_2             input.txt.save.2    processid.c
Week1_2.c           linux              signal.c
childusingfork       linux.c            simplecommandloop
childusingfork.c     multiplechild       simplecommandloop.c
client              multiplechild.c     sjf
client.c            myfifo             sjf.c
concurrently        myfifo.c           sjf.c.save
concurrently.c      myprocessid        skill3a
doublefork          myprocessid.c       skill3a.c
doublefork.c        myprocessid.c.save  skill3b
fcfs                orphanprocess       skill3b.c
fcfs.c              orphanprocess.c     skill6
fifo                output.txt          skill6.c
fifo.c              parent             socket
fifo_read           parent.c           socket.c
fifo_read.c         parent.save        username
fifo_write          parentchild        username.c
fifo_write.c        parentchild.c      wait
file_write.c        parentwaitsforchild wait.c
fork                parentwaitsforchild.c welcome
fork.c              posix.c.save        welcome.c
fork1               practical1          zombie
fork1.c             practical1.c        zombie.c
hello               practical2
bindu@Bindusree:~$

```